Transformer From Scratch: A Line-by-Line Technical Review

Understanding Self-Attention, Layer Normalization,

and Backpropagation Using Pure NumPy

Andrew Bieber

Computer Engineering

March 2026

Repository:

github.com/andrew9990828/first_transformer

This document accompanies a complete from-scratch Transformer implementation and serves as a learning and teaching artifact.

Purpose of This Document

This document is a detailed technical review of a minimal Transformer language model implemented entirely from scratch using pure NumPy. The purpose of this report is not to present a novel architecture or achieve state-of-the-art performance, but to develop a first-principles understanding of how Transformers function at the level of individual operations, tensor shapes, and gradient flow.

This review represents the transition from implementation to comprehension. Each section revisits a component of the model line-by-line, pairing code with intuition, diagrams, and mathematical reasoning. The end goal is to be able to explain the full forward and backward pass of a Transformer block clearly and without reliance on machine learning frameworks.

Model Overview

This project implements a minimal Transformer-based language model trained to perform next-token prediction on raw text. The model operates at the character level and learns to predict the most likely subsequent character given a fixed-length context window. Despite its simplicity, the model exhibits core Transformer behaviors such as context-dependent attention, structured token representations, and stable gradient-based learning.

Architecturally, the model consists of a single Transformer block with one self-attention head, followed by a feed-forward network and an output projection into vocabulary space. Token embeddings and learned positional embeddings are combined to form the model input, which then flows through self-attention, residual connections, layer normalization, and a small multilayer perceptron. The final output is a distribution over the vocabulary for each token position, trained using softmax cross-entropy loss.

Several constraints were imposed intentionally to prioritize understanding over scale. The model uses a small embedding dimension, a short context window, a single attention head, and plain stochastic gradient descent. No automatic differentiation or machine learning frameworks are employed. Every forward pass, backward pass, and parameter update is written explicitly in NumPy, exposing the exact tensor operations and gradient flow that underpin modern Transformer architectures.

At a high level, training proceeds by repeatedly sampling short subsequences from a longer text corpus, predicting the next token at each position, computing the resulting loss, and propagating gradients backward through the model to update parameters. Over time, this process allows the model to internalize local statistical patterns, syntactic structure, and stylistic cues present in the training data.

Day 1 — Data, Vocabulary, and Tokenization

Objective

Our first task was to turn raw text into tokens. We do this by first identifying individual characters, mapping an individual ID to each unique element, and representing the entire text as a 1D sequence of token IDs.

Reading the Corpus

```
with open('data_textfiles/the_raven_edgarallanpoe.txt', 'r',
encoding="utf-8") as file:
    content = file.read()
```

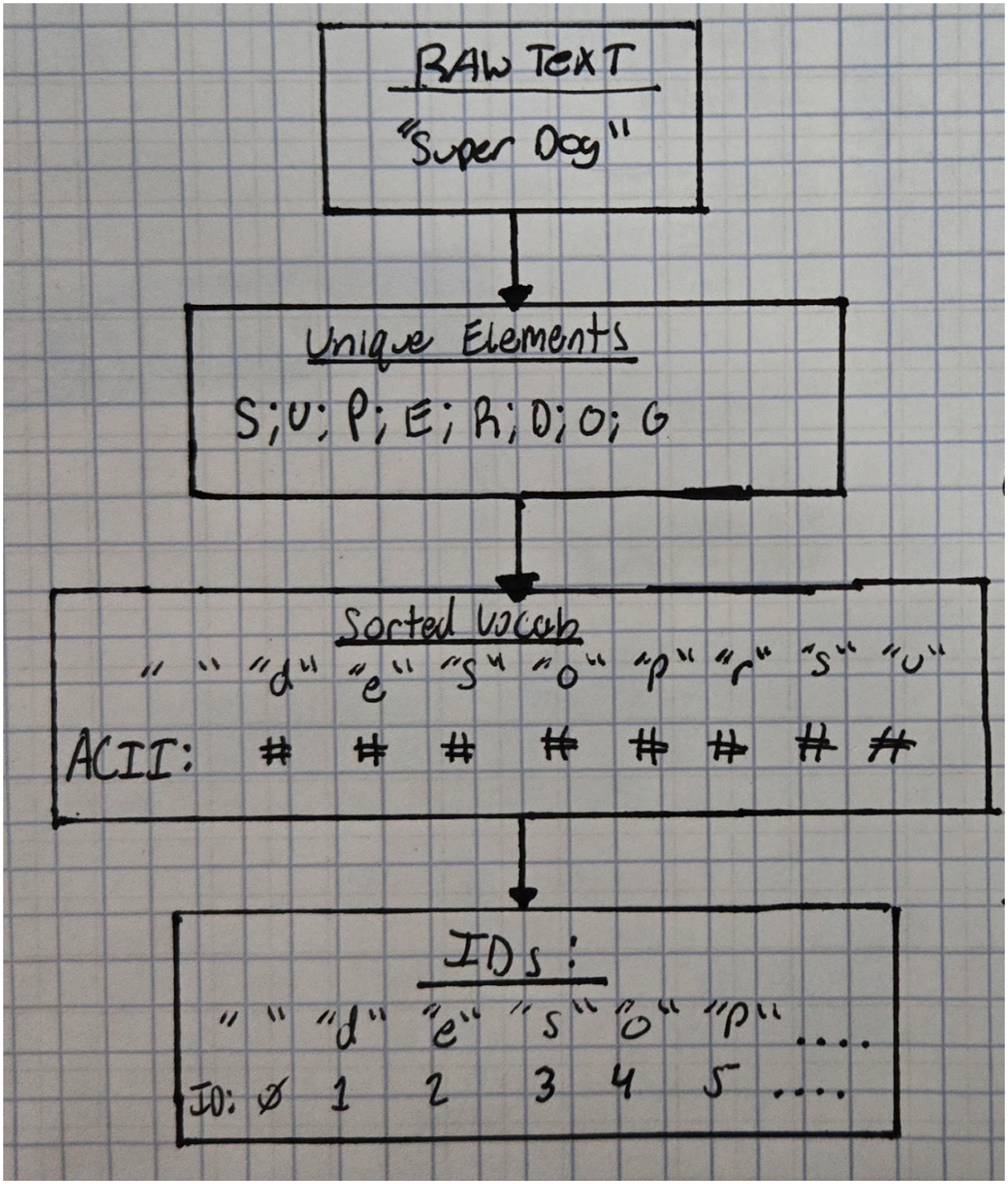
First, we must read the entire contents of the file into our program. We don't care about sentences or lines at this stage because we need to just identify all of the unique characters in this entire text file. Character level modeling simplifies assumptions because we don't need to classify ourselves what counts as a word, a sentence, or even a paragraph.

We are simply going to be predicting the next character to be placed in the sequence. This simplifies parsing and everything else significantly and I purposefully kept it with next char prediction for these reasons.

Vocabulary Construction

```
# Use pythons set function to store unique elements automatically
unique_chars = set()
for ch in content:
# Store all unique elements
unique_chars.add(ch)
# Find the length of the list of unique elements
num_of_chars = len(unique_chars)
# Store the chars as ordered using pythons sorted() function
vocab = sorted(unique_chars)
```

Next, we had to use the set() function which returned all the unique characters in the text file. Then we loop through those unique elements and assign them a number or "ID". Finally, we use sorted() to sort the IDs and properly assign each unique character with a numerical representation making it deterministic. The length of our unique element set is essential later for softmax and parameter sizes. The whole point of this step was to have determinism from sorting.



Character ↔ ID mappings

```
# Create mappings
char_to_idx = {}
idx_to_char = {}
```

```
# Loop through to map everything
for idx, char in enumerate(vocab):
    char_to_idx[char] = idx
    idx_to_char[idx] = char
```

Here we use dictionaries. Dictionaries are an O(1) operation in python because they are implemented as a hashtable. The IDs have no semantics because they just represent a single individual char. One ID just represents one unique element at this point.

Bidirectional mapping later is needed so that way we can both train the model and generate our outputs. Without bidirectional mapping we just wouldn't be able to do both and can't show the model is actually learning anything.

Encoding the Text

```
encoded = []
for ch in content:
    encoded.append(char_to_idx[ch])
encoded_txt = np.array(encoded, dtype=int)
```

Tokenization is simply mapping a symbol to a text. So in this case we are just mapping each element in the poem to one ID. So the result is the poem represented in IDs and then stored in an array labeled "encoded_txt".

Tokenization is assigning symbols. Later, embeddings assign meaning. This encoded text is entirely static and cannot be trained on because we haven't gotten to embeddings yet!

Train / test split

```
# Split data first: To actually show learning
# Did this inside of the NN as well.
train_size = int(0.9 * len(encoded_txt))
train_data = encoded_txt[:train_size]
test_data = encoded_txt[train_size:]
```

Although the data is sequential, a simple contiguous split is still appropriate in this setting. The model is trained only on subsequences sampled from the training portion of the text and never sees characters from the test region during optimization. Generalization here means the ability to predict plausible next characters in unseen portions of the sequence using patterns learned earlier, rather than memorizing specific character runs.

This split therefore provides a basic but effective check that the model is learning transferable structure rather than reproducing the training data verbatim.

Key Takeaways

- Raw text must first be converted into a numerical form before any learning can occur. In this model, characters are treated as the fundamental tokens to simplify assumptions and avoid the need for word-level parsing.

- Vocabulary construction identifies all unique characters in the corpus and assigns them deterministic integer IDs. These IDs are stable labels and carry no semantic meaning.

- Tokenization, in this context, is the process of mapping each character in the text to its corresponding integer ID, producing a 1D sequence of tokens that represents the entire corpus.

- Token IDs are not embeddings. They are discrete identifiers and cannot be directly used for learning, gradient computation, or similarity comparisons.

- A simple train/test split remains meaningful for sequential data because the model is trained only on subsequences from the training region and never observes the test portion during optimization.

- Generalization in a character-level language model refers to the ability to predict plausible next characters in unseen portions of the sequence based on learned statistical patterns rather than memorization.

Day 2 — Model Dimensions, Learnable Parameters, and Embeddings

Objective

The goal of this section was to declare the shapes necessary for our model. If we don't explicitly make this part clear, we can't do the math later. We have to initialize every single **learnable parameter** once **outside** of the training loop before the model is trained. If we re-initialized inside of the training loop, all learning inside of the model would be lost.

Hyperparameters

```
# Define sizes
V = len(vocab)      # size 57, vocab contains total # of unique Chars
# C is the embedding dimension (I chose 32)
C = 32
# Set B and T
B = 12
T = 8
# Define H
H = C
# Define the hidden layer H_ff
# 4 * C is a standard choice
H_ff = 4 * C
```

V (Vocabulary size)

V represents the number of unique characters present in the input text. This defines the discrete input space of the model, as every character must map to exactly one token ID in the range [0, V). V therefore determines the size of the token embedding table and the output vocabulary used during prediction.

C (Embedding dimension)

C is the embedding dimension. Each token ID is mapped to a learnable vector of length C. In this implementation, C = 32, meaning each character is represented by a 32-dimensional vector. At initialization these values are arbitrary, but during training they become structured representations that the model can reason over.

B (Batch size)

B represents the number of independent sequences processed in parallel during training. Here, B = 12, meaning the model simultaneously processes 12 separate subsequences sampled from the text. B exists purely for computational efficiency and does not change the model's behavior.

T (Sequence length / context window)

T is the length of each sequence in a batch. In this model, T = 8, meaning each training example consists of 8 consecutive characters. The model learns to predict the next character at each position using only the previous characters within this fixed context window.

(B, T, C) — the core tensor shape

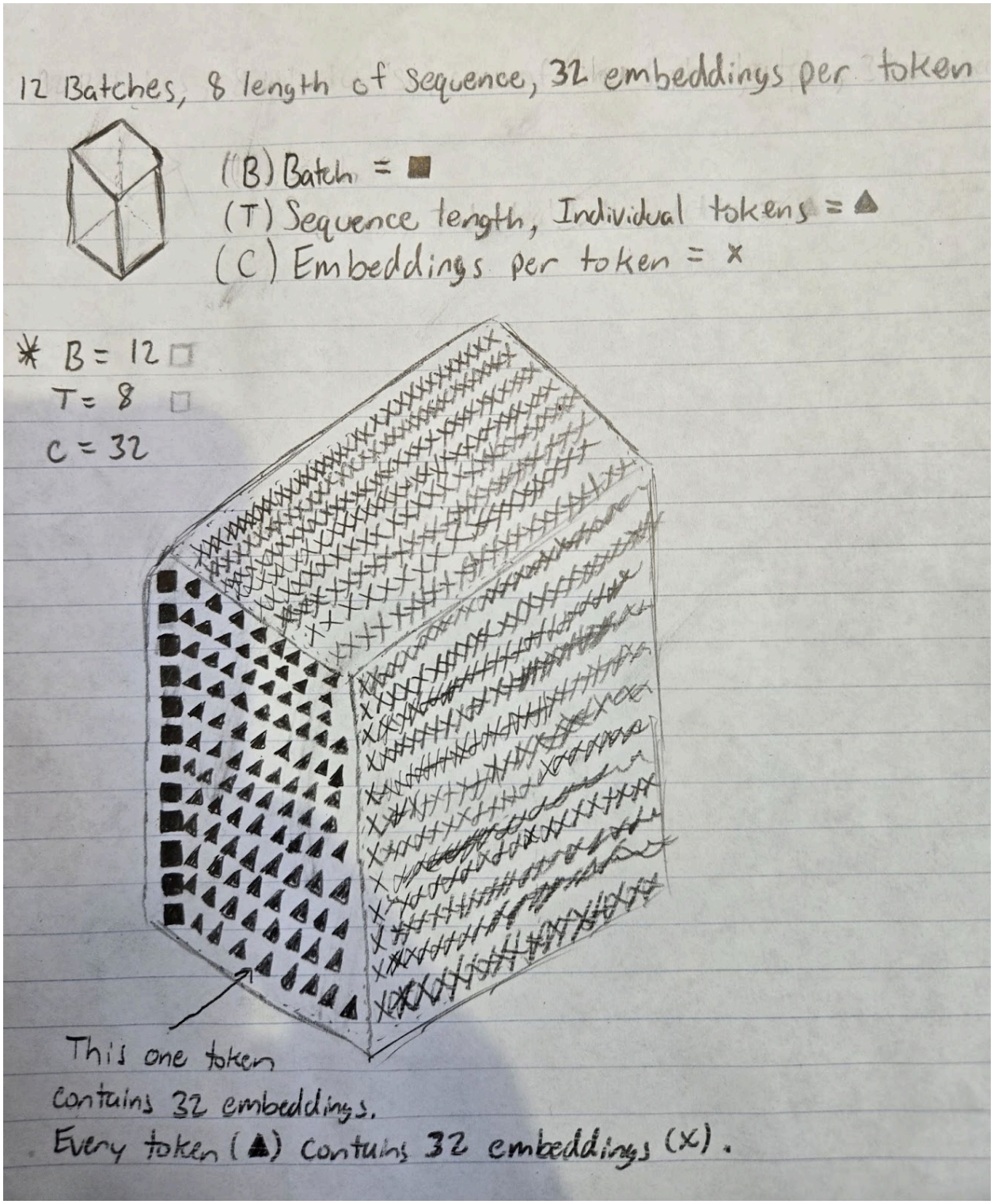
After embedding lookup and positional encoding, the model operates on tensors of shape (B, T, C). This shape is preserved through attention, normalization, and feed-forward layers, forming the backbone of all subsequent Transformer computations.

H (Attention head size)

H is the dimensionality of the attention projections. In this implementation, H = C, meaning each token's query, key, and value vectors have the same dimensionality as the embeddings. This simplifies the model and makes attention operate directly over embedding space.

H_ff (Feed-forward hidden width)

H_ff = 4C represents the hidden width of the feed-forward network. This follows standard Transformer design and allows the model to temporarily expand the representation capacity before projecting back down to C. This parameter is used later and does not affect the data flow at this stage.


12 Batches, 8 length of sequence, 32 embeddings per token
(B) Batch = ■
(T) Sequence length, Individual tokens = ▲
(C) Embeddings per token = x
* B = 12
T = 8
C = 32
This one token
contains 32 embeddings.
Every token (▲) contains 32 embeddings (x).

What “parameters” are in this project

```
# Create the token embedding table
# Shape is (57, 32) meaning each unique element
# V=57 unique tokens, each gets a 32-dim vector.
token_embedding_table = np.random.uniform(-0.1, 0.1, size=(V, C))
# Create the positional embedding table with shape (T,C)
positional_embedding_table = np.random.uniform(-0.1, 0.1, size=(T,C))
positions = np.arange(T)
```

The **token embedding table** is a learnable parameter matrix of shape (V,C), where V is the vocabulary size and C is the embedding dimension. Each row corresponds to a unique token (character) and stores a C-dimensional continuous vector. During training, these vectors are updated via backpropagation and come to represent semantic information about each token. At runtime, token IDs are used to *index* into this table to retrieve their embeddings.

The **positional embedding table** is a learnable parameter matrix of shape (T,C), where T is the sequence length. Since self-attention is permutation-invariant, positional embeddings inject explicit information about token order into the model. Each position in the sequence is assigned its own C-dimensional vector, which is added to the corresponding token embedding.

Token IDs themselves are **not learnable** and carry no meaning beyond identification. Meaning emerges only after mapping tokens to continuous embedding vectors. These embeddings form the foundational representation that enables attention, softmax, and gradient-based learning.

Why random initialization is necessary

If all learnable parameters were initialized to zero, every embedding and weight would evolve identically during training, creating a symmetry problem that prevents meaningful learning. Random initialization breaks this symmetry by giving each token and parameter a unique starting point, allowing the model to learn distinct representations. Small initialization ranges are used to keep early activations and gradients stable, reducing the risk of exploding values during the initial stages of training.

Attention projection matrices (Q, K, V)

```
# Initialize weights
Wq = np.random.uniform(-0.1, 0.1, size=(C, H))
Wk = np.random.uniform(-0.1, 0.1, size=(C, H))
Wv = np.random.uniform(-0.1, 0.1, size=(C, H))
```

The matrices **Wq**, **Wk**, and **Wv** project token embeddings into **query**, **key**, and **value** spaces. These projections are later used by the attention mechanism to determine how strongly each token should attend to others. All three matrices have shape **(C, H)**. Although Q, K, and V originate from the same input embeddings, learning separate projections allows the model to assign different semantic roles to each representation.

LayerNorm parameters (gamma/beta)

```
# LayerNorm parameters
gamma1 = np.ones(C)
beta1  = np.zeros(C)
gamma2 = np.ones(C)
beta2  = np.zeros(C)
```

Layer normalization uses learnable **scale (γ)** and **shift (β)** parameters, each with shape **(C,)**. These parameters allow the model to normalize activations while preserving expressive power. LayerNorm improves training stability and helps prevent exploding or vanishing activations during optimization.

Feedforward network weights (the MLP inside the block)

```
# Set weights and biases with out input and hidden_dim set
# w1 shape(32, 128)
weight1 = np.random.randn(C, H_ff) * np.sqrt(2 / C)
# w1 shape(128, 32)
# For the initialization here, we need to have np.sqrt(2 / H_ff)
# because this happens after ReLU
weight2 = np.random.randn(H_ff, C) * np.sqrt(2 / H_ff)
# b1 shape(128,)
bias1 = np.zeros((H_ff,), dtype=np.float32)
# b2 shape(32,)
bias2 = np.zeros((C,), dtype=np.float32)
```

Each transformer block includes a two-layer feedforward network applied independently at every sequence position. The hidden dimension is expanded to **H_ff = 4C**, then compressed back to **C**, introducing nonlinearity and additional representational capacity.

- **Weight1:** (C, H_ff)
- **Bias1:** (H_ff,)
- **Weight2:** (H_ff, C)
- **Bias2:** (C,)

He initialization is used for these layers to maintain stable variance through the ReLU activation.

Output projection to vocabulary

```
# Output projection weights
W_vocab = np.random.uniform(-0.1, 0.1, size=(C, V))
b_vocab = np.zeros((V,), dtype=np.float32)
```

The output projection maps the final hidden representation at each position into **logits over the vocabulary**. These logits are later converted into probabilities using softmax.

- **W_vocab:** (C, V)
- **b_vocab:** (V,)

This is the final step that allows the model to predict the next token.

Causal mask (why it exists)

```
causal_mask = np.triu(np.ones((T, T)), k=1).astype(bool)
```

In autoregressive language modeling, the model must not attend to future tokens. The causal mask enforces this constraint by blocking attention to positions **t+1 through T−1** when predicting token **t**. Masked positions are assigned large negative values before softmax, causing their attention weights to become effectively zero.

ReLU (activation function)

```
def relu(x):
    return np.maximum(0, x)
```

ReLU introduces nonlinearity by preserving positive activations and zeroing negative values. This allows the model to learn more expressive representations and helps maintain stable gradients during training.

Key Takeaways

- **Model behavior is entirely determined by its parameters.** Embedding tables, projection matrices, normalization parameters, and feedforward weights are the learnable components that transform token IDs into predictions.

- **Dimensionality choices define the geometry of the model.**
    - V defines the size of the vocabulary and therefore the output space.
      C defines the embedding dimension and is the core feature width of the model.
    - (B, T, C) describes how data flows through the model: batches of sequences, each represented by vectors in embedding space.

- **Token and positional embeddings serve distinct roles.** Token embeddings represent *what* the symbol is, while positional embeddings encode *where* it appears in the sequence. Both are required for attention to function meaningfully.

- **Attention projections (Q, K, V) are learned linear transformations.** Although they originate from the same embeddings, separate projection matrices allow the model to compute relationships between tokens from different perspectives.

- **The feedforward network expands and compresses information.** Using a wider hidden dimension (H_ff = 4C) allows the model to apply nonlinearity before returning to embedding space, increasing expressive power.

- **Layer normalization stabilizes training.** Learnable scale (gamma) and shift (beta) parameters ensure activations remain numerically stable throughout forward and backward passes.

- **Causal masking enforces autoregressive behavior.** Preventing tokens from attending to future positions ensures the model learns to predict the next token using only past context.

- **Random initialization is essential for learning.** Initializing parameters with small random values breaks symmetry and enables meaningful gradient-based optimization.

Day 3 — Training Loop, Batch Sampling, and Optimization Flow

Objective

The goal of this section is to formalize **how the Transformer learns**, not by introducing new architectural components, but by defining the **training process itself**. This includes how data is sampled, how predictions are evaluated, and how parameter updates are applied iteratively over time.

This section marks the shift from *defining the model* to *teaching the model*.

Why a Training Loop Is Necessary

Similarly to a neural network or any machine learning model, transformers do not magically learn after the data passes through it once. For learning to happen, a transformer must see the dataset numerous times. Learning occurs from passing the data through the set, passing it through an MLP (multi-layered-perceptrons), calculating probabilities (softmax), backpropagation (what parameter contributed to the losses?), loss calculation (cross-entropy; etc), and finally optimization. To simplify it further the model needs to take an “educated guess”, see what caused it to be so wrong, and then use that information to make another, better “educated guess” the next time. Think of it as learning from failure, because that's exactly what it is. So, each iteration through the transformer slightly adjusts parameters in the direction that it’s likely to be more correct. The entire goal is less prediction error in each pass.

Epochs, Steps, and Learning Rate

```
num_epochs = 100
steps_per_epoch = 10000
learning_rate = 0.01
# The entire training loop is encapsulated inside of the epoch loop
# and the steps loop.
for epoch in range(num_epochs):
    epoch_loss = 0.0

    for step in range(steps_per_epoch):
```

One “epoch” is one entire pass through the training loop. One “step” is one batch update. The learning rate controls how aggressively we change our parameters later. A low learning rate is

much slower than a faster one; however, it is very stable and there is little noise as it continues to decrease in loss (noise or deviations is completely normal later in the training). A larger learning rate changes the speed at which we learn, but it's very unstable. So for this project I just picked 0.01, a relatively small learning rate.

Batch Sampling from Sequential Data

```
# Sample batch from TRAIN data
        # Randomize to be "i must be ≤ N - (T+1)""
        N = len(train_data)
        max_start = N - (T+1)

        inputs = []
        targets = []

        for _ in range(B):
            # Set random start point in the sequence of tokens
            i = np.random.randint(0, max_start + 1)

            x = train_data[i : i+T]
            y = train_data[i+1 : i+T+1]

            inputs.append(x)
            targets.append(y)

        inputs = np.stack(inputs, axis=0)
        targets = np.stack(targets, axis=0)
```

The dataset used for training is a single long sequence of tokenized characters. To train the model using a fixed context window, we must extract **random contiguous subsequences** from this sequence rather than processing the entire text at once. Each training sample is therefore a short, continuous slice of the original text, and the starting position of each slice is randomized to expose the model to diverse local contexts.

Each training batch consists of **B = 12 independent samples**. Every sample is a sequence of **T = 8 consecutive characters** drawn from the poem. Although the underlying data is sequential, these subsequences are sampled independently, allowing the model to learn statistical patterns without relying on absolute position within the text.

To prevent indexing beyond the bounds of the dataset, the range of valid starting indices must be constrained. If the total number of tokens in the training data is N, the maximum valid starting

index is N−(T+1). This value is stored as max_start and ensures that both the input sequence and its corresponding target sequence can be safely extracted.

For each sample in the batch, a random starting index i is selected from this valid range. The **input sequence** is defined as the slice:

x=train_data[i:i+T]

and the corresponding **target sequence** is defined as:

y=train_data[i+1:i+T+1]

This one-token offset implements *teacher forcing*, where the model is trained to predict the next character at each position in the sequence. By comparing the model's predictions against the target tokens, the training process can determine how accurate each prediction was and compute a loss value that guides parameter updates during optimization.

Why Random Sampling Is Used

Random sampling is very important for this model. By randomly sampling, this prevents memorization of fixed positions in the text. We don't want the model to memorize the text as is, we want the model to learn the style with how Edgar Allen Poe wrote poetry. When we randomly sample, the same character shows up in many different contexts in the corpus. With random sampling, the model is learning statistical structure, not its absolute location. Although the text is sequential, learning benefits from stochastic exposure to varied contexts.

Batch Tensor Construction

Just to clarify, the shapes for our inputs and our targets should be (B, T). Just like how we defined our parameters earlier, these shapes both consist of 12 batches (B) and a sequence of 8 characters per batch (T). At this point in training these are just IDs, we haven't assigned embeddings just yet which is important because embeddings give the tokens meaning. So at this point we have the data sampled but there has been no embeddings and no gradients exist yet.

Key Takeaways

- Sequential text data can be treated as a training distribution by randomly sampling **contiguous subsequences**, allowing a fixed-length context window to be used without processing the entire corpus at once.

- Random sampling prevents the model from memorizing absolute positions in the text. Instead, tokens are encountered in many different local contexts, encouraging the model to learn **statistical structure and style** rather than fixed character locations.

- Each training batch consists of multiple independent samples, even though they originate from the same underlying sequence. This independence improves learning stability and acts as a form of stochastic regularization.

- Careful control of sampling bounds is required when working with sequential data. Defining a valid maximum starting index ensures that both input and target sequences can be safely constructed without indexing errors.

- Input and target tensors at this stage contain only **token IDs**, not embeddings. They represent symbolic information only and do not yet carry semantic meaning or gradients.

- The offset between input and target sequences implements *teacher forcing*, enabling the model to compare its predictions against the correct next token at every position and compute an informative loss signal during training.

- Batch construction marks the final preprocessing step before learning begins. Once inputs are embedded and passed through the model, gradients can be computed and parameters updated through backpropagation.

Day 4 — Token Embeddings, Positional Encoding, and Single-Head Self-Attention

Objective

The objective of this section is to transform discrete token IDs into continuous, trainable representations and to compute context-aware token interactions using single-head self-attention. By the end of this section, we will have produced contextualized embeddings that encode both token identity and sequence-level relationships.

From Token IDs to Trainable Embeddings

```
# Now do the indexing with the batch we had
# Should create a tensor now with (B, T, C) as the shape.
tok = token_embedding_table[inputs]
```

When we sampled the text in the previous section, we only got 12 batches of a sequence of 8 characters that were symbolized by IDs. So the shape before we apply embeddings is (B, T). Here we are taking the inputs (B, T) and adding the embeddings (C) to the tokens.

So by adding the embeddings to these individual tokens, we are assigning **meaning** to them. This creates a tensor of shape (B, T, C) with B representing the 12 batches, T representing the 8 individual characters per batch, and C representing the 32 embeddings per individual character. So we turned out tokens into a representation of its embeddings by looking them up in our token_embedding_table we created before training.

Positional Encoding and Sequence Order

```
pos = positional_embedding_table[positions]
# Now we combine the positional embeddings to
# the token embeddings
x_seq = tok + pos
```

Self-attention by itself has no concept of order. Without positional information, the model would treat a sequence as a bag of tokens and wouldn’t know which characters came first or last. To fix this, we assign each position in the sequence its own learnable embedding vector.

These positional embeddings are added directly to the token embeddings, so each token carries both its identity and its position in the sequence. This allows the attention mechanism to reason

about order while keeping the math and tensor shapes simple. *"Token embeddings tell the model **what** a character is. Positional embeddings tell the model **where** it shows up."*

Linear Projections into Query, Key, and Value Spaces

```
#   Query (Q) → what am I looking for?
#   Key (K) → what do I contain?
#   Value (V) → what information do I offer?
Q = x_seq @ Wq
K = x_seq @ Wk
V_att = x_seq @ Wv
```

Even though every token starts as the same embedding vector space, attention needs tokens to play different roles. A token sometimes needs to *ask* a question, sometimes it needs to *describe itself*, and sometimes it needs to *pass information forward*. That's why we create three separate representations: Query, Key, and Value.

- **Query (Q)**: what this token is currently looking for
- **Key (K)**: what this token contains / advertises
- **Value (V)**: the actual information this token will contribute if attended to

Every token produces **all three** — it both asks questions and offers information at the same time. The following coded matrices let the model learn *what to pay attention to*, *how to be matched*, and *what information to pass along*.

At this point, the input tensor has shape (B, T, C), meaning we have B independent sequences, each containing T tokens represented by C-dimensional vectors. The projection matrices Wq, Wk, and Wv each have shape (C, H), so they take a token embedding and re-express it in a new H-dimensional space. After applying these linear transformations, the resulting Query, Key, and Value tensors all have shape (B, T, H). The sequence length stays the same, and tokens are not interacting with each other yet. We are simply creating three different views of each token that will later be used by the attention mechanism.

Token-to-Token Similarity via Dot Products

```
# K: (B, T, H)  ->  K_T: (B, H, T)
K_T = np.transpose(K, (0, 2, 1))
# Q: (B, T, H) @ K_T: (B, H, T)  ->  scores: (B, T, T)
scores = Q @ K_T
```

Since the shape of our key is (B, T, H) we need to transpose the tensor. We want every token's query vector to dot with every token's key vector. Transposing K makes the dimensions line up so the dot product happens across H. At a very high level, think of rotating a puzzle piece so it fits the other corresponding pieces.

Scores come from the dot products between the query tensor with the transposed key tensor. The scores (B, T, T) or (B, i, j) means that *in these batches, how much does each token 'i' cares about every other token 'j' based on the query and key.* The shape becomes (B, T, T) because each token is scoring against all T tokens. **Each token creates T scores — one score for every token in the sequence, including itself.**

Score Scaling and Numerical Stability

```
# Scale attention scores for numerical stability
scores = scores / np.sqrt(H)
```

So this doesn't blow up the Q and K gradients, we add score scaling. Since each query and key is length **H**, the dot product naturally increases in magnitude as H increases. If we don't apply score scaling, large dot products make softmax extremely sharp. This means one token gets almost all the attention and every other token goes to near zero. Gradients can vanish and training would become extremely slow.

Dividing by the square root of (H) normalizes the variance of the dot product, keeps values in a reasonable range, and preserves relative importance between scores. This scaling does not change *which* tokens should be attended to — it only keeps the numbers well-behaved during training.

Causal Masking for Autoregressive Prediction

```
scores_masked = scores.copy()
# causal_mask is (T, T). This line broadcasts over batch dimension (B)
scores_masked[:, causal_mask] = -1e9   # effectively -inf for softmax
```

When predicting the next token, the model must not be allowed to see future tokens. Without masking, token t could attend to any other token ahead of it in the sequence. This would leak future information and allow the model to “cheat” its learning.

This casual mask has the shape (T, T) with each row representing the current token and each column being the tokens it can attend to. Future positions are always blocked. The causal mask ensures each token can only attend to itself and all tokens that come before it.

When we do this, we don’t remove the future values, we just make them impossible to attend to by filling those spots with (-1e9) which is just an extremely large negative number. Therefore the tokens marked with this value contribute a negligible amount to attention (Effectively, nearly, zero). Masking happens **before** softmax so that blocked positions receive zero probability.

Softmax Normalization of Attention Weights

```
# Softmax calculated here. Inspired from my NN froms scratch
# This is the softmax over attention scores
scores_max = np.max(scores_masked, axis=-1, keepdims=True)
scores_exp = np.exp(scores_masked - scores_max)
sum_exp = np.sum(scores_exp, axis=-1, keepdims=True)
attention = scores_exp / sum_exp
```

Softmax turns raw attention scores into probabilities. Before this step, scores could be any real number; negative, position, large, small. This isn’t interpretable. After softmax values become any number between 0 and 1. Each row or sequence must sum up to 1.

The model is determining how much of that 1 it's going to give each token. This is deciding how much attention to give each individual token in a sequence. Each row of the attention matrix represents how one token distributes its focus across the sequence.

It is important to know here that softmax is being applied to the last dimension, or the last T in (B, T, T). This means that each token gets its own probability distribution. Each token independently decides where to focus.

Subtracting scores_max matters because exponentials blow up fast. Subtracting the max keeps the numbers small without changing the result. This does not change the output of softmax — it only prevents numerical overflow. The resulting attention matrix tells us how much each token attends to every other token in the sequence. At this point, the model has decided **where** to look — the next step determines **what information** to pull in.

Weighted Aggregation of Values

```
# Weighted sum outputs
# This produces the actual output of attention
out = attention @ V_att
```

Each token pulls in information from other tokens based on the attention weight:

- attention tells the model **how much** to focus on each token
- V_att contains the **actual information** each token can contribute
- The matrix multiply combines them into a single vector per token

Tokens don't pick just one other token — they blend information from many:

- High attention weight → more influence
- Low attention weight → less influence
- Masked tokens → zero influence

After this step, each token embedding contains information about the entire sequence, filtered by attention.

Key Takeaways

- Token IDs are purely symbolic and cannot be learned from directly. Embedding lookup converts these discrete IDs into continuous vectors that the model can reason over and update through gradients.

- Positional embeddings are required because self-attention has no inherent sense of order. Adding positional embeddings allows each token to carry information about **where** it appears in the sequence, not just **what** it is.

- Query, Key, and Value are three learned views of the same token embedding. Queries represent what a token is looking for, keys represent what a token contains, and values represent the information a token can contribute if attended to.

- Self-attention begins when each token's query is compared against every token's key using dot products. This produces a token-to-token similarity matrix of shape (B, T, T), where each token scores every other token in the sequence.

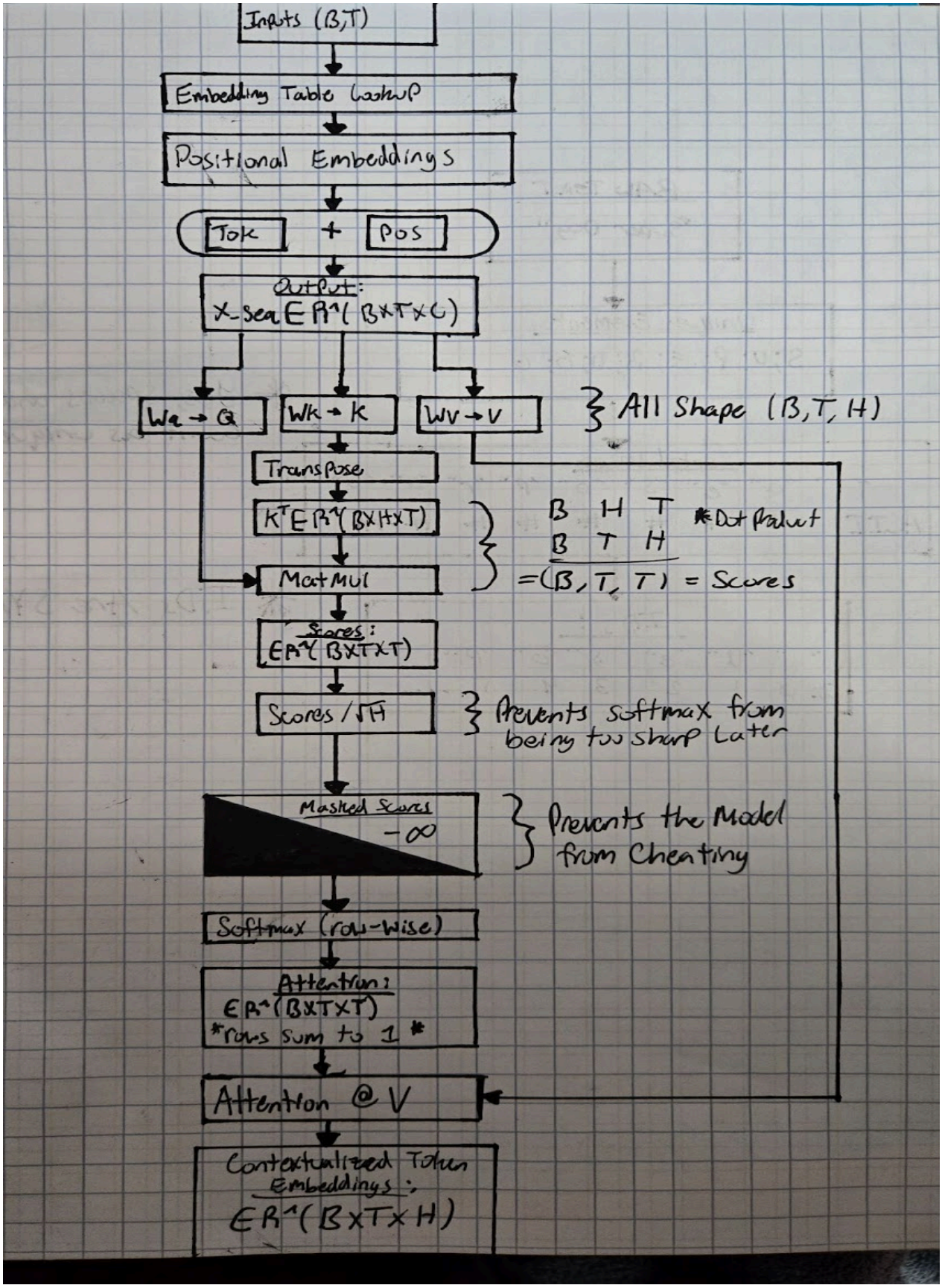
- Scaling attention scores by √H is necessary for numerical stability. Without scaling, dot products grow too large, causing softmax to become overly sharp and gradients to vanish during training.

- Causal masking enforces autoregressive behavior by preventing tokens from attending to future positions. Masked positions are assigned extremely negative values so they receive zero probability after softmax.

- Softmax converts raw attention scores into normalized probabilities. Each token independently decides how to distribute its attention across the sequence, with attention weights summing to one.

- The final attention output is produced by taking a weighted sum of the value vectors. This step blends information from multiple tokens, producing context-aware representations for each position.

- After single-head self-attention, tokens are no longer independent. Each token embedding now contains information about the entire sequence, filtered through learned attention patterns.


Inputs (B,T)
Embedding Table Lookup
Positional Embeddings
Tok + Pos
Output:
X_seq ∈ ℝ^(B×T×C)
Wq → Q
Wk → K
Wv → V
All Shape (B,T,H)
Transpose
K^T ∈ ℝ^(B×H×T)
B H T
B T H
*Dot Product
MatMul
=(B, T, T) = Scores
Scores:
∈ ℝ^(B×T×T)
Scores / √H
Prevents softmax from being too sharp Later
Masked Scores
−∞
Prevents the Model from Cheating
Softmax (row-wise)
Attention:
∈ ℝ^(B×T×T)
rows sum to 1
Attention @ V
Contextualized Token Embeddings:
∈ ℝ^(B×T×H)

Day 5 — Residual Connections, Layer Normalization, and the Feed-Forward Network

Objective

In this section, we complete the forward pass of the transformer block. At this point attention outputs are now stabilized, transformed, and finalized. It is important to note that the shape of (B, T, C) remains constant throughout this process.

Residual Connection After Self-Attention

```
# The residual connection
x1 = x_seq + out
```

The residual layer is a connection that adds the input layer (x_seq) directly to its output (out). This prevents vanishing gradients and preserves information, allowing it to directly propagate.

Layer Normalization (Post-Attention)

```
# Applied after the residual connection to scale the stabilization of
activations.
mean = np.mean(x1, keepdims=True, axis=-1)
var = np.var(x1, keepdims=True, axis=-1)
center = x1 - mean
normalize = center/np.sqrt(var + 1e-5)
scale = gamma1 * normalize + beta1
```

Layer normalization ensures that the residual connection is within a reasonable range and that gradients remain stable throughout the activations later. Layer normalization is token-wise and happens across the embedding dimension. We prevent numerical issues in the future, speeds up convergence, and the data distribution remains stable across all layers.

Feed-Forward Network (MLP)

```
# The input is our token length
input_dim = C
# Implement the forward pass
```

```
Z1 = scale @ weight1 + bias1
A1 = relu(Z1)
# ff_out has shape (12, 8, 32)
ff_out = A1 @ weight2 + bias2
```

The forward pass after self-attention is complementary. It builds context and allows each token to independently process, transform, and store knowledge. This build consisted of two linear layers and ReLU as the activation function. Our weights consist of H_ff which was equal to 4*C. So H_ff expands the weight to be 128, Then the output weight returns to our base shape of (12, 8, 32).

Second Residual Connection

```
# Residual connection
x2 = scale + ff_out
```

We add the output of the MLP to the previous output of the layernorm post-attention. This is our 2nd residual connection and again the same process is being used with layernorm. The residual connection fixes our issues of vanishing gradients while the 2nd layernorm prevents training from blowing up and stabilizes our data during training.

Second Layer Normalization

```
# Layer Normalization
# Same pattern as earlier
var2 = np.var(x2, keepdims=True, axis=-1)
mean2 = np.mean(x2, keepdims=True, axis=-1)
center2 = x2 - mean2
normalize2 = center2/np.sqrt(var2 + 1e-5)
scale2 = gamma2 * normalize2 + beta2
```

Similarly to the first time we did layernorm, this is the exact same process and formula as before. We simply have to ensure our residual connection is in a reasonable range and we do this by feeding our 2nd residual connection.

End-of-Block Summary

At this stage, the forward pass of a single Transformer block is complete. The tensor scale2 represents the final output of the block and preserves the core shape (B, T, C). Despite multiple transformations, additions, and nonlinearities, the embedding dimension and sequence structure remain intact, making the block modular and composable.

Each token representation has undergone a structured sequence of refinements. First, token and positional embeddings were combined to encode identity and order. Self-attention then allowed each token to gather contextual information from previous tokens in the sequence. The first residual connection preserved the original signal while incorporating contextual updates, and layer normalization stabilized the resulting activations.

The feed-forward network then applied nonlinear, position-wise transformations, expanding the representation into a higher-dimensional space before compressing it back to the embedding dimension. A second residual connection preserved the post-attention signal, and the final layer normalization ensured numerical stability of the block's output.

The resulting tensor now contains context-aware, nonlinearly transformed, and stabilized token embeddings. Each token vector encodes both its identity and relevant information from prior tokens in the sequence. This representation is now ready to be projected into vocabulary space for prediction, followed by loss computation and gradient-based learning.

Key Takeaways

- Residual connections preserve information flow and improve gradient stability by allowing the model to refine representations instead of replacing them entirely.

- Layer normalization stabilizes activations by normalizing each token independently across the embedding dimension, preventing exploding or vanishing values during training.

- The feed-forward network increases expressive power by applying nonlinear, position-wise transformations without mixing information between tokens.

- The Transformer block preserves the fundamental tensor shape (B, T, C), enabling modular stacking and architectural scalability.

- After completing this block, token representations are contextualized, numerically stable, and ready for output projection, loss computation, and backpropagation.

Day 6 — Output Projection, Softmax, and Cross-Entropy Loss

Objective

The state of the transformer of scale2 must be translated to raw scores. These are known as logits and these are raw data (**NOT probabilities**). Our goal with these logits is to turn these numbers into probabilities, and then use those probabilities to calculate the loss of the model before we use backpropagation.

Shapes to keep in mind:

- Scale2 : (B, T, C)
- Logits: (B, T, V)
- Probs: (B, T, V)
- Loss: Scalar

Output Projection to Vocabulary (Logits)

```
logits = scale2 @ W_vocab + b_vocab
```

This line is giving each individual token a score for every possible next character. The mapping of the math looks like this:

(B,T,C) @ (C,V) → (B,T,V)

This is just a linear classifier applied at every time position.

Softmax Over Vocabulary Dimension

```
max_logits1 = np.max(logits, axis=-1, keepdims=True)
shifted1 = logits - max_logits1          # numerical stability
exp_scores1 = np.exp(shifted1)
sum_exp1 = np.sum(exp_scores1, axis=-1, keepdims=True)

probs = exp_scores1 / sum_exp1           # (B, T, V)
```

We already did softmax for self-attention earlier, but we must turn the logits from the forward pass into probabilities. We do that by applying softmax again. The formula for softmax remains completely the same as earlier, just this time its dealing with logits, not the output from self-attention. It is important to note that the probabilities are applied over the last dimension (**axis = -1**) and each batch that has a row of a sequence now becomes a probability distribution. Important to also note we subtract from the max for numerical stability as well.

Cross-Entropy Loss (Negative Log-Likelihood)

```
b_idx = np.arange(B)                    # batch indices
t_idx = np.arange(T)                    # time indices
```

At this stage, probs has shape (B, T, V) and represents the probability distribution over the vocabulary at each batch and time position. However, we do not compare entire probability distributions when computing the loss. Instead, we only care about the probability assigned to the correct next token at each (b, t) position.

The targets tensor contains integer token IDs of shape (B, T). These IDs tell us which vocabulary index is the correct answer for each position. The advanced indexing operation:

```
# Select the probability of the correct target token at each (b, t)
correct_probs = probs[b_idx[:, None], t_idx[None, :], targets]
```

extracts the model's assigned probability for the correct token at every batch and time position. The result has shape (B, T).

We then compute the negative log-likelihood:

```
# Cross-entropy loss (negative log-likelihood)
loss = -np.mean(np.log(correct_probs + 1e-9))
```

Taking the logarithm transforms multiplicative probabilities into additive contributions, which makes optimization more stable and mathematically convenient. The negative sign ensures that higher probabilities for correct predictions reduce the loss. If the model assigns high probability to the correct token, the loss decreases. If it assigns low probability, the loss increases sharply.

The small constant 1e-9 prevents numerical issues when the probability is extremely close to zero, avoiding log(0).

What This Loss Represents (Interpretation)

The scalar loss represents how well the model predicts the correct next tokens across the entire batch and sequence.

More specifically:

- If the model is confident and correct (probabilities close to 1 for true tokens), the loss is low.
- If the model is uncertain or wrong (probabilities spread out or low for true tokens), the loss is high.
  If the model assigns extremely low probability to correct tokens, the loss grows large.

This loss is averaged across all (B × T) positions, meaning every token prediction contributes equally to the final scalar value.

In language modeling, this is equivalent to maximizing the likelihood of the training data under the model. Minimizing cross-entropy corresponds to increasing the probability assigned to the true next tokens.

At this point, the entire forward computation has been reduced to a single scalar value. That scalar measures prediction error and serves as the objective function for learning.

Bridge to Backprop

Up until now, the model has only performed forward computations. It has transformed token IDs into contextual embeddings, projected them into vocabulary space, converted logits into probabilities, and measured error using cross-entropy.

The loss scalar is the starting point for backpropagation.

Backpropagation will compute:

- The gradient of the loss with respect to the logits
- Then with respect to the output projection weights
- Then through the second layer normalization
- Through the MLP
- Through residual connections
- Through attention

- And finally into embeddings

Every parameter in the model will receive a gradient indicating how it contributed to the prediction error.

The next section will reverse the entire forward pass, computing derivatives step-by-step to update the model's parameters using gradient descent.

Key Takeaways

- The Transformer block output must be projected into vocabulary space to produce logits for every possible next token.
- Softmax converts logits into valid probability distributions independently at each batch and time position.
- Cross-entropy loss measures how much probability the model assigns to the correct next tokens.
- The loss is a single scalar that summarizes prediction error across the entire batch.
- Numerical stability techniques (subtracting the max, adding epsilon) are essential to prevent overflow and undefined logarithms.
- The computed loss serves as the starting point for backpropagation through the entire Transformer.

Day 7 — Backpropagation (Part I): Loss, Output Projection, and LayerNorm2

Objective

The goal of this section is to begin reversing the Transformer block.

Up to this point, the model has performed only forward computations. It has:

- Embedded tokens
- Computed attention
- Applied residual connections
- Normalized activations
- Passed representations through an MLP
- Projected into vocabulary space
- Computed cross-entropy loss

All of that ultimately reduced to a single scalar: loss.

Backpropagation begins at that scalar and propagates gradients backward through every operation that produced it. In this section, we will compute gradients for:

1. Softmax + Cross-Entropy
2. Output projection (linear layer)
3. Second LayerNorm and its residual connection

By the end of this section, gradients will be flowing into both:

- scale (the post-LN1 tensor)
- ff_out (the MLP output)

This marks the true beginning of gradient flow inside the Transformer block.

Step 11 (Backward) — Softmax + Cross-Entropy

Forward Recap

- logits: (B, T, V)
- probs = softmax(logits)
- loss = mean(-log(probs[target]))

The forward pass converts logits into probabilities and then measures how much probability mass was assigned to the correct tokens.

Now we compute the derivative of the loss with respect to the logits.

Key Result

When softmax and cross-entropy are combined, the gradient simplifies to:

$$\frac{\partial L}{\partial logits} = \frac{probs - one_hot(targets)}{B \cdot T}$$

This simplification is extremely important.
It eliminates the need to compute the full Jacobian of softmax.

Implementation

```
# Number of independent prediction sites (loss is averaged)
Npos = B * T

# Start from probabilities
dlogits = probs.copy()                          # (B, T, V)

# Subtract 1 at the correct vocab index for each (b, t)
# This is equivalent to subtracting a one-hot vector,
# but avoids explicitly constructing one.
dlogits[b_idx[:, None], t_idx[None, :], targets] -= 1.0
# Account for mean reduction in the loss
 dlogits /= Npos
```

Interpretation

This gradient has clear meaning:

- If the model assigned too much probability to an incorrect token → decrease it.
- If the model assigned too little probability to the correct token → increase it.
- Every token position contributes equally because we divide by B*T.

At this stage, we now know how the logits must change to reduce the loss.

Step 10 (Backward) — Output Projection (Linear Layer)

Forward

$$logits = scale2@W_{vocab} + b_{vocab}$$

Shapes:

- scale2: (B, T, C)
- W_vocab: (C, V)
- logits: (B, T, V)

This is a standard linear layer applied independently at each token position.

Backward Responsibilities

We must compute:

1. Gradient of bias
2. Gradient of weight matrix
3. Gradient flowing back into scale2

Bias Gradient

Each vocabulary bias term affects every batch and time position equally.

```
# Bias affects every (b,t) equally → sum over batch + time
db_vocab = np.sum(dlogits, axis=(0, 1))         # (V,)
```

Shape: (V,)

Weight Gradient

We flatten (B, T) into one dimension.

Each token position acts like an independent training example.

```
# Flatten (B,T) so each token position is treated as
# an independent training example for the linear layer
N = B * T
X = scale2.reshape(N, C)                          # (BT, C)
G = dlogits.reshape(N, V)                         # (BT, V)
# Weight gradient follows standard linear layer rule
dW_vocab = X.T @ G                                # (C, V)
```

Shape: (C, V)

This follows the standard linear-layer gradient rule:

$$dW = X^T \cdot dY$$

Gradient into Transformer Representation

```
# Gradient flowing back into transformer representation
dscale2 = dlogits @ W_vocab.T                   # (B, T, C)
```

Shape: (B, T, C)

Now gradients have flowed back into the final Transformer block representation.

The vocabulary projection layer is fully reversed.

Step 9 (Backward) — LayerNorm2 + Residual Connection

Forward Recap

```
# Forward:
```

```
#   x2      = scale + ff_out
#   scale2  = LayerNorm(x2)
```

So the computation was:

1. Residual addition
2. Layer normalization

Backward must reverse those two operations.

Gradients for Gamma and Beta

LayerNorm applies:

$$scale2 = \gamma_2 \cdot normalize2 + \beta_2$$

So:

```
# Gamma and beta gradients:
#   beta shifts → sum of gradients
#   gamma scales normalized input
dbeta2  = np.sum(dscale2, axis=(0,1))                # (C,)
dgamma2 = np.sum(dscale2 * normalize2, axis=(0,1))    # (C,)
```

- beta shifts → sum of gradients
- gamma scales normalized input → multiply then sum

Shape: (C,)

Backprop Through Normalization

LayerNorm enforces:

- Zero mean
- Unit variance
- Per token

Because normalization couples dimensions together, its backward formula is dense.

Core implementation:

```
# Gradient through affine LN step
dxhat = dscale2 * gamma2                                # (B, T, C)

# Core LayerNorm backward formula
dx2 = (1.0 / Cdim) * std_inv * (
      Cdim * dxhat
      - np.sum(dxhat, axis=-1, keepdims=True)
      - xhat * np.sum(dxhat * xhat, axis=-1, keepdims=True)
)
```

Conceptually, this step:

- Removes gradient components that would change the mean
- Removes gradient components that would change the variance
- Restores gradient in original coordinate system

After this step:

dx2 has shape (B, T, C)

Residual Split

Forward was:

$$x2 = scale + ff_out$$

Addition distributes gradients equally to both branches:

```
# Residual split:
# x2 = scale + ff_out
dscale  = dx2
dff_out = dx2
```

This is one of the most important structural ideas in Transformers:

Residual connections cause gradients to split and flow down multiple paths.

End-of-Day Summary

At this stage:

- The loss has been differentiated.
- The vocabulary projection layer has been reversed.
- The second LayerNorm has been reversed.
- The residual connection has split gradients into two branches.

We now have gradients flowing into:

- dscale (input to MLP branch)
- dff_out (output of MLP)

The Transformer block is partially reversed.

The next section will propagate gradients through:

- The feed-forward network
- The first LayerNorm
- Self-attention

Key Takeaways

- Softmax + cross-entropy simplifies dramatically during backpropagation.
- Linear layer backward requires flattening token positions into independent samples.
- LayerNorm backward is complex because normalization couples dimensions.
- Residual connections duplicate gradient flow across branches.
- Backpropagation mirrors the forward pass in exact reverse order.

Day 8 — Backpropagation (Part II): MLP Backward + LayerNorm1 Backward + Residual Split

Objective

In the previous section, gradients were propagated from the scalar loss backward through the vocabulary projection layer, through LayerNorm2, and through the second residual connection. At that point, gradients reached two tensors inside the Transformer block:

- dff_out, the gradient flowing into the output of the feed-forward network
- dscale, the gradient flowing into the output of LayerNorm1

The purpose of this section is to continue reversing the Transformer block. Specifically, we will:

1. Fully reverse the feed-forward network (MLP)
2. Reverse LayerNorm1
3. Split gradients through the first residual connection

By the end of this section, gradients will be flowing into both:

- x_seq (the embedding pathway)
- out (the attention pathway).

Self-attention backpropagation will be handled in the next section.

STEP 8 (BACKWARD): Feed-Forward Network (MLP)

Forward Recap

The feed-forward network was defined as:

$$Z1 = scale@weight1 + bias1$$

$$A1 = ReLU(Z1)$$

$$ff_out = A1@weight2 + bias2$$

The tensor shapes were:

- scale: (B, T, C)

- weight1: (C, H_ff)
- Z1: (B, T, H_ff)
- weight2: (H_ff, C)
- ff_out: (B, T, C)

At this stage in backpropagation, we already have:

- dff_out with shape (B, T, C)

Reversing the Second Linear Layer

We begin by reversing the final linear transformation:

$$ff_out = A1@weight2 + bias2$$

Each vocabulary feature in bias2 affects every token position equally, so its gradient is obtained by summing across batch and time dimensions.

```
# Bias gradient: sum over all token positions

db2 = np.sum(dff_out, axis=(0,1))                    # (C,)
```

Next, we compute the gradient for weight2. Although tensors are 3-dimensional, the MLP is applied independently at every token position. Conceptually, each (b, t) pair behaves like an independent training example. To apply the standard linear-layer gradient rule cleanly, we reshape:

$$(B, T, \cdot) \rightarrow (B \cdot T, \cdot)$$

After flattening, we apply the familiar rule:

$$dW = X^T \cdot dY$$

```
# Flatten tokens to compute weight gradient

A_flat = A1.reshape(BT, H_ff)                        # (BT, H_ff)

G_flat = dff_out.reshape(BT, C)                      # (BT, C)

dW2 = A_flat.T @ G_flat                              # (H_ff, C)
```

Finally, we compute the gradient flowing backward into the hidden activations:

$$dA1 = dff_out@weight2^T$$

```
# Gradient flowing back into hidden activation

dA1 = dff_out @ weight2.T                                # (B, T, H_ff)
```

At this stage, dA1 has shape (B, T, H_ff).

ReLU Backward

ReLU is elementwise and simple: it passes gradient only where the forward activation was positive.

$$dZ1 = dA1 \odot \mathbf{1}(Z1 > 0)$$

```
# ---- ReLU backward ----
# Gradient only flows where activation was positive
dZ1 = dA1 * (Z1 > 0)                                     # (B, T, H_ff)
```

This enforces that neurons inactive during the forward pass receive no gradient signal.

Reversing the First Linear Layer

We now reverse:

$$Z1 = scale@weight1 + bias1$$

As before, the bias gradient is computed by summing across batch and time dimensions.

```
# Bias gradient

db1 = np.sum(dZ1, axis=(0,1))                            # (H_ff,)
```

The weight gradient is computed by flattening again and applying:

$$dW1 = scale^T \cdot dZ1$$

```
# Weight gradient (flatten again)
```

```
S_flat  = scale.reshape(BT, C)                          # (BT, C)

G1_flat = dZ1.reshape(BT, H_ff)                         # (BT, H_ff)

dW1 = S_flat.T @ G1_flat                                # (C, H_ff)
```

Finally, we compute the gradient flowing back into scale through the MLP branch:

$$dscale_{mlp} = dZ1@weight1^T$$

```
# Gradient flowing back into 'scale' from MLP branch

dscale_mlp = dZ1 @ weight1.T                            # (B, T, C)
```

Gradient Merge at the Residual Junction

The tensor scale influences the loss through two different paths:

1. Through the second residual connection (x2 = scale + ff_out)
2. Through the feed-forward network itself

Because of this, the total gradient flowing into scale is the sum of gradients from both paths:

$$dscale = dscale + dscale_{mlp}$$

```
# Merge gradients at residual junction

# scale participates in BOTH:

#   - residual path

#   - MLP path

dscale = dscale + dscale_mlp
```

This is a structural property of residual networks: whenever a tensor is used in multiple downstream computations, its gradients accumulate.

At this point, the feed-forward network has been fully reversed.

STEP 7 (BACKWARD): LayerNorm1 + Residual1

Forward Recap

Earlier in the forward pass:

$$x1 = x_seq + out$$

$$scale = LayerNorm(x1)$$

We now have:

- dscale with shape (B, T, C)

We must compute:

- Gradients for gamma1 and beta1
- The gradient flowing into x1
- The residual split into dx_seq and dout

Gradients for Gamma1 and Beta1

LayerNorm applies an affine transformation after normalization:

$$scale = \gamma_1 \odot \hat{x}_1 + \beta_1$$

Thus, beta1 accumulates gradients directly, while gamma1 scales the normalized input before summation.

```
# gamma1/beta1 gradients

dbeta1  = np.sum(dscale, axis=(0, 1))                # (C,)

dgamma1 = np.sum(dscale * normalize, axis=(0, 1))    # (C,)
```

Backprop Through the Affine Step

$$dxhat_1 = dscale \odot \gamma_1$$

```
# Backprop through LN1 affine
dxhat1 = dscale * gamma1                                  # (B, T, C)
```

Backprop Through Normalization

LayerNorm enforces per-token zero mean and unit variance across the embedding dimension. Because normalization couples feature dimensions, its backward pass is dense.

Using the cached forward quantities (normalize, var, and eps), we compute:

$$dx1 = \text{LayerNormBackward}(dxhat_1)$$

```
# Cached forward quantities from LN1

xhat1 = normalize                                         # (B, T, C)

std_inv1 = 1.0 / np.sqrt(var + eps)                       # (B, T, 1)


# LayerNorm backward formula (same as LN2, different cached vars)

dx1 = (1.0 / Cdim) * std_inv1 * (

          Cdim * dxhat1

          - np.sum(dxhat1, axis=-1, keepdims=True)

          - xhat1 * np.sum(dxhat1 * xhat1, axis=-1, keepdims=True)

     )                                                    # (B, T, C)
```

After this step, `dx1` has shape (B, T, C).

Residual Split

From the forward definition:

$$x1 = x_seq + out$$

The gradient distributes equally to both inputs:

$$dx_seq = dx1$$

$$dout = dx1$$

```
# Residual split for x1 = x_seq + out

dx_seq = dx1          # gradient into x_seq from residual

dout   = dx1          # gradient into attention output "out"
```

This completes the reversal of:

- The feed-forward network
- LayerNorm1
- The first residual connection

Gradients are now positioned to flow backward into:

- The embedding pathway (x_seq)
- The attention mechanism (out)

Self-attention backpropagation will be addressed next.

End-of-Day Summary

In this section, the Transformer block was reversed through its feed-forward and normalization components. The MLP was fully differentiated, producing gradients for both linear layers and their biases. Gradients into scale were merged from multiple computational paths, reflecting the structure introduced by residual connections.

LayerNorm1 was then reversed, restoring gradients into x1 while computing parameter updates for gamma1 and beta1. Finally, the residual addition split gradients into two branches, preparing the model for attention-level backpropagation.

The Transformer block has now been reversed up to the attention mechanism.

Key Takeaways

- Backpropagation through the MLP is mechanically identical to a standard 2-layer neural network applied independently at every token position. The only difference is that the input and gradients live in 3D tensors, so reshaping (flattening B×T) makes the linear-layer gradient rules clean and consistent.

- The feed-forward network contributes gradients to **two places at once**:

    - to its own parameters (W1,b1,W2,b2)

    - back into the Transformer stream through its input (scale)
      This reinforces a core idea: **whenever a tensor influences the loss through multiple downstream paths, its gradients accumulate by summation.**

- ReLU backward is purely a gating operation. It does not "create" gradient — it only **routes** it through activations that were positive in the forward pass and blocks it everywhere else.

- LayerNorm backward is dense because normalization couples all features within the embedding dimension. Unlike ReLU or residual addition, a change in one feature affects mean and variance, so the gradient must correct for both:

    - components that would shift the mean

    - components that would change the variance

- LayerNorm introduces its own learnable parameters γ and β, and backprop explicitly shows their roles:

    - β collects gradients directly (it shifts activations)

    - γ collects gradients weighted by the normalized activations (it scales the normalized signal)

- Residual connections are not just a forward-pass trick — they are a **gradient routing mechanism**. During backprop, a residual addition *splits* gradients cleanly into both

branches. This is why deep Transformer stacks remain trainable.

- After reversing MLP + LayerNorm1 + Residual1, gradients now flow into two critical streams:
    - dx_seq → the embedding pathway (token + positional embeddings)
    - dout → the attention output pathway
    - This is the precise point where the model becomes “attention-dependent” in the backward pass.
- At the end of Day 2 of backprop, the Transformer block has been fully reversed **up to** self-attention. The only remaining task to complete the full backward pass is differentiating the attention mechanism itself (Q/K/V projections, masking, softmax attention weights, and value aggregation).

Day 9 — Backpropagation (Part III): Self-Attention and Embeddings

Objective

The purpose of this section is to complete the reversal of the Transformer block by differentiating the self-attention mechanism and propagating gradients back into the embedding tables.

In previous sections, gradients were propagated backward through:

- The output projection
- LayerNorm2
- The feed-forward network
- LayerNorm1
- The first residual connection

At this point, gradients exist in two streams:

- dout — flowing into the attention output
- dx_seq — flowing through the residual embedding pathway

This section will:

1. Reverse value aggregation (out = attention @ V)
2. Reverse softmax attention weights
3. Reverse attention score scaling
4. Reverse the Q/K dot product interaction
5. Reverse the Q/K/V projections
6. Accumulate gradients into x_seq
7. Scatter gradients back into embedding tables

After this section, the entire Transformer block will be fully differentiated.

Step 6 (Backward) — Self-Attention

Forward Recap

Self-attention was computed as:

$$Q = x_seqW_q$$

$$K = x_seqW_k$$

$$V = x_seqW_v$$

$$scores = \frac{QK^T}{\sqrt{H}}$$

$$attention = softmax(mask(scores))$$

$$out = attention \cdot V$$

Shapes:

- Q, K, V: (B, T, H)
- scores: (B, T, T)
- attention: (B, T, T)
- out: (B, T, H)

Unlike the MLP, attention mixes information across tokens, so its backward pass propagates gradients between sequence positions.

Backprop Through Value Aggregation

```
V_T = np.transpose(V_att, (0, 2, 1))                # (B, H, T)
dattention = dout @ V_T                              # (B, T, T)

att_T = np.transpose(attention, (0, 2, 1))          # (B, T, T)
dV_att = att_T @ dout                               # (B, T, H)
```

Each token output is a weighted sum of value vectors.

Gradients split into two components:

$$\frac{\partial L}{\partial attention} = dout \cdot V^T$$

$$\frac{\partial L}{\partial V} = attention^T \cdot dout$$

Key Insight:

This is the only place where gradients flow from later tokens back into earlier ones via attention weights.

Backprop Through Softmax

```
dscores_masked = attention * (
            dattention - np.sum(dattention * attention, axis=-1,
keepdims=True)
        )                                                        # (B,T,T)

# zero out gradients for masked positions (future tokens)
dscores_masked[:, causal_mask] = 0.0
```

Attention weights were computed row-wise:

$$a = softmax(s)$$

Softmax backward simplifies to:

$$ds = a \odot (da - \sum(da \odot a))$$

Important:

Masked entries were constants (−1e9).
Gradients for those positions must be zeroed explicitly.

Conceptual meaning:

Softmax backward redistributes gradients so that increasing one attention weight decreases others in the same row.

Backprop Through Score Scaling

Forward:

$$scores = raw_scores/\sqrt{H}$$

Backward:

```
dscores = dscores_masked / np.sqrt(H)                    # (B, T, T)
```

$$d(raw_scores) = dscores/\sqrt{H}$$

Scaling ensures numerical stability in forward pass and must be reversed accordingly.

Backprop Through Dot Product

```
dQ = dscores @ K                                          # (B, T, H)
dscores_T = np.transpose(dscores, (0, 2, 1))              # (B, T, T)
dK = dscores_T @ Q                                        # (B, T, H)
```

This is a batched matrix multiplication.

Gradients:

$$dQ = dscores \cdot K$$

$$dK = dscores^T \cdot Q$$

Key Insight:

This is where cross-token gradient coupling happens.
Every token's query influences every token's key.

Attention backward is dense across the sequence dimension.

Backprop Through Projections

```
BT = B * T
X_flat = x_seq.reshape(BT, C)                                # (BT, C)

dQ_flat = dQ.reshape(BT, H)                                   # (BT, H)
dK_flat = dK.reshape(BT, H)                                   # (BT, H)
dV_flat = dV_att.reshape(BT, H)                             # (BT, H)

# Weight gradients
dWq = X_flat.T @ dQ_flat                                      # (C, H)
dWk = X_flat.T @ dK_flat                                      # (C, H)
dWv = X_flat.T @ dV_flat                                      # (C, H)

# Gradient back into x_seq from each projection path
dx_from_Q = dQ @ Wq.T                                         # (B, T, C)
dx_from_K = dK @ Wk.T                                         # (B, T, C)
dx_from_V = dV_att @ Wv.T                                   # (B, T, C)

# Total gradient into x_seq combines:
#   - residual path (dx_seq from Step 7)
#   - projection paths (Q/K/V)
dx_seq = dx_seq + dx_from_Q + dx_from_K + dx_from_V         # (B, T, C)
```

Forward:

$$Q = x_seqW_q$$

$$K = x_seqW_k$$

$$V = x_seqW_v$$

Each projection is a standard linear layer.

Gradients:

$$dW_q = X^T dQ$$

$$dW_k = X^T dK$$

$$dW_v = X^T dV$$

$$dx_from_Q = dQW_q^T$$

$$dx_from_K = dKW_k^T$$

$$dx_from_V = dVW_v^T$$

Because x_seq participates in three projections, gradients accumulate:

$$dx_seq = dx_seq + dx_from_Q + dx_from_K + dx_from_V$$

This accumulation is another structural property of multi-branch computation graphs.

Step 5 (Backward) — Embeddings

```
# Gradient into tok and pos from x_seq = tok + pos
dtok = dx_seq                                                  # (B, T, C)
dpos = np.sum(dx_seq, axis=0)                                   # (T, C)
# ------------------------------------------------------------
# Backprop into positional_embedding_table
# ------------------------------------------------------------
dpositional_embedding_table = np.zeros_like(positional_embedding_table)  #
(T, C)
dpositional_embedding_table[positions] += dpos
```

Forward:

$$x_seq = tok + pos$$

Gradients distribute:

$$dtok = dx_seq$$

$$dpos = \sum_{batch} dx_seq$$

Positional embeddings are broadcast across the batch dimension, so their gradients must be summed over B.

Scatter-Add into Token Embeddings

```
dtoken_embedding_table = np.zeros_like(token_embedding_table)  # (V, C)
# Efficient scatter-add: add dtok[b,t] into row token_id = inputs[b,t]
np.add.at(dtoken_embedding_table, inputs, dtok)
```

Token embeddings were selected by integer indexing:

$$tok = token_embedding_table[inputs]$$

Gradients do not flow into integer IDs.
Instead, they accumulate into the rows of the embedding table that were selected.

This requires scatter-add.

Conceptually:

Every occurrence of a token contributes to updating its embedding vector.

End-of-Day Summary

At this stage:

- The entire Transformer block has been reversed.
- Every parameter now receives a gradient.
- Gradients have flowed from:
    - Loss
    - Vocabulary projection
    - LayerNorm2
    - MLP

- LayerNorm1
- Self-attention
- Q/K/V projections
- Embedding tables

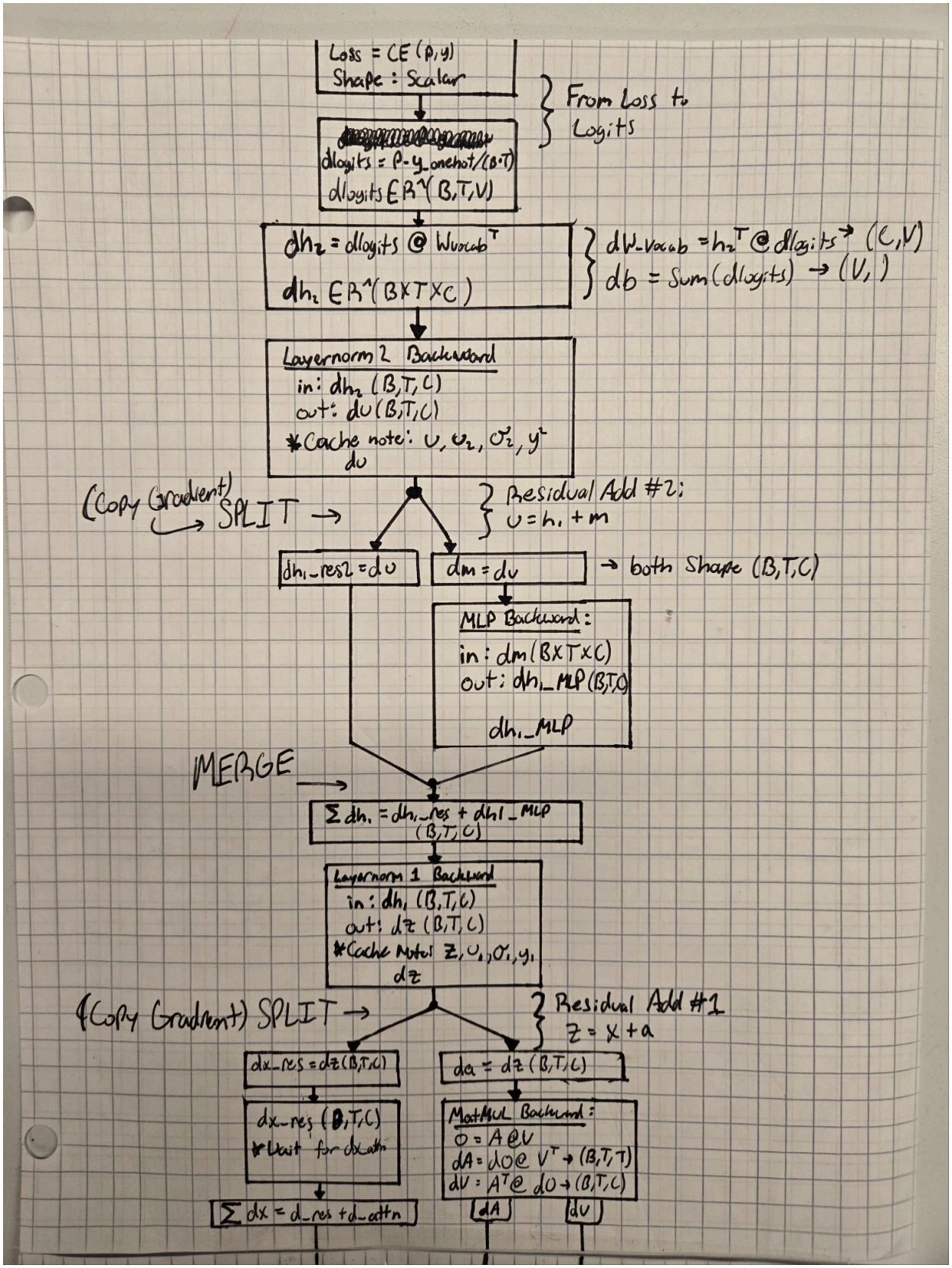
No computational path remains undifferentiated.

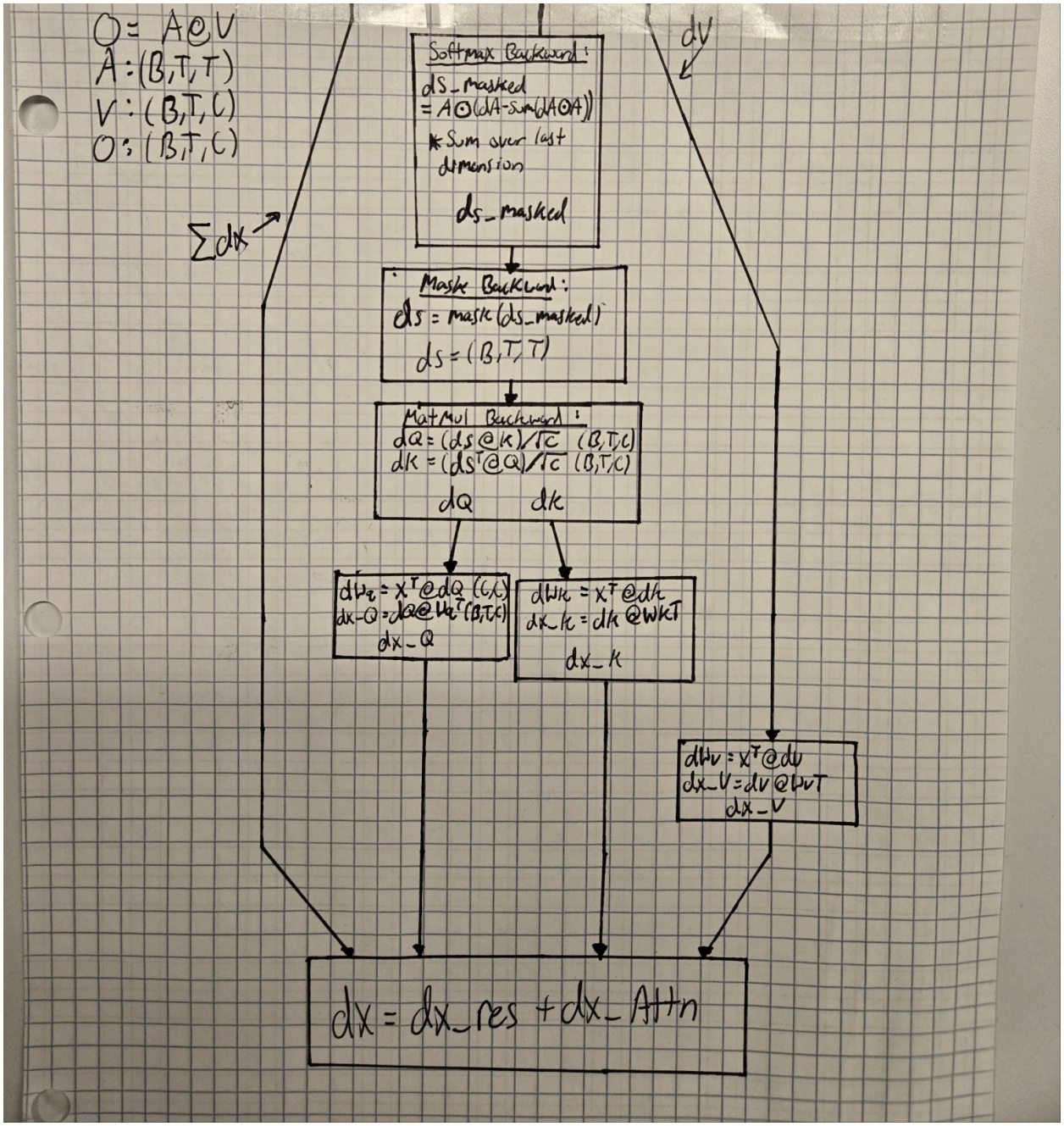
The Transformer block is now fully invertible under gradient descent.

Key Takeaways

- Self-attention backward is structurally more complex than the MLP because it couples tokens across the sequence dimension.
- Softmax backward redistributes gradients within each token's attention distribution.
- Masked attention entries must have zero gradient.
- Q/K dot products create dense gradient interactions between tokens.
- Projection layers behave like standard linear layers once reshaped.
- Embedding layers require scatter-add to accumulate gradients correctly.
- After this section, the Transformer block is fully differentiated end-to-end.

Entire Backprop Block Diagram


Loss = CE(p, y)
Shape : Scalar
From Loss to Logits
dlogits = p - y_onehot/(B·T)
dlogits ∈ ℝ^(B,T,V)
dh2 = dlogits @ Wvocab^T
dh2 ∈ ℝ^(B×T×C)
dW_vocab = h2^T @ dlogits → (C,V)
db = Sum(dlogits) → (V,)
Layernorm 2 Backward
in: dh2 (B,T,C)
out: du (B,T,C)
*Cache note: u, σ2, y^2
du
(Copy Gradient) SPLIT →
Residual Add #2:
u = h1 + m
dh1_res2 = du
dm = du
→ both Shape (B,T,C)
MLP Backward:
in: dm (B×T×C)
out: dh1_MLP (B,T,C)
dh1_MLP
MERGE →
Σ dh1 = dh1_res + dh1_MLP
(B,T,C)
Layernorm 1 Backward
in: dh1 (B,T,C)
out: dz (B,T,C)
*Cache Note: z, u1, σ1, y1
dz
(Copy Gradient) SPLIT →
Residual Add #1
z = x + a
dx_res = dz (B,T,C)
da = dz (B,T,C)
dx_res (B,T,C)
*Wait for dx_attn
MatMul Backward:
O = A @ V
dA = dO @ V^T → (B,T,T)
dV = A^T @ dO → (B,T,C)
Σ dx = d_res + d_attn
dA
dV


O = A@V
A : (B,T,T)
V : (B,T,C)
O : (B,T,C)
Softmax Backward:
dS_masked
= A⊙(dA-sum(dA⊙A))
* Sum over last dimension
ds_masked
dV
Σdx
Mask Backward:
ds = mask(ds_masked)
ds = (B,T,T)
MatMul Backward:
dQ = (ds@k)/√C (B,T,C)
dk = (ds^T@Q)/√C (B,T,C)
dQ
dk
dWq = x^T@dQ (C,C)
dx_Q = dQ@Wq^T (B,T,C)
dx_Q
dWk = x^T@dk
dx_k = dk@WkT
dx_k
dWv = x^T@dv
dx_V = dv@WvT
dx_V
dx = dx_res + dx_Attn

Day 10 — Optimization: Stochastic Gradient Descent (SGD) Parameter Updates

Objective

At this point, every learnable parameter in the Transformer has a computed gradient. The purpose of SGD is to **apply those gradients** to slightly adjust each parameter in the direction that reduces the loss.

This step is the bridge between "we computed derivatives" and "the model actually learns."

What SGD Does

For any parameter θ, SGD performs:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta L$$

Where:

- η = learning rate
- ∇θL = gradient of loss w.r.t. that parameter
- Subtracting the gradient moves parameters **downhill** on the loss surface.

Parameters Updated in This Project

After backprop, we update all learnable tensors, including:

Embeddings

- token_embedding_table (V, C)
- positional_embedding_table (T, C)

Attention projections

- Wq, Wk, Wv (C, H)

LayerNorm parameters

- gamma1, beta1 (C,)
- gamma2, beta2 (C,)

Feed-forward (MLP)

- weight1 (C, H_ff), bias1 (H_ff,)
- weight2 (H_ff, C), bias2 (C,

Output projection

- W_vocab (C, V), b_vocab (V,)

Implementation (Plain NumPy SGD)

The update rule is identical for every parameter:

Wq -= learning_rate * dWq

In practice, you repeat this for all parameters:

```
# Update all parameters using computed gradients
token_embedding_table -= learning_rate * dtoken_embedding_table
positional_embedding_table -= learning_rate * dpositional_embedding_table

Wq -= learning_rate * dWq
Wk -= learning_rate * dWk
Wv -= learning_rate * dWv

gamma1 -= learning_rate * dgamma1
beta1 -= learning_rate * dbeta1

weight1 -= learning_rate * dW1
```

```
bias1 -= learning_rate * db1
weight2 -= learning_rate * dW2
bias2 -= learning_rate * db2

gamma2 -= learning_rate * dgamma2
beta2 -= learning_rate * dbeta2

W_vocab -= learning_rate * dW_vocab
b_vocab -= learning_rate * db_vocab

epoch_loss += loss
```

Why Gradients Must Be Reset Each Step

Gradients represent **only the current batch's** contribution to the loss. If you accumulate gradients across steps unintentionally, you are no longer doing SGD on a single batch update.

In a pure NumPy implementation, this typically means re-initializing your d* arrays each training step (or overwriting them cleanly).

Key Takeaways

- Backprop computes "how to change parameters." SGD actually **changes them**.
- The SGD rule is simple and uniform across the model: subtract learning rate times gradient.
- After the update, the next forward pass uses slightly improved parameters, and loss should trend downward over many steps.

Analogies to Key Transformer Concepts

Tokenization (character-level)

- Like turning a sentence into **individual Lego bricks**. You're not interpreting meaning yet—you're just standardizing pieces you can work with.

Vocabulary + lookup tables

- Like giving every Lego brick a **barcode** (an ID). The model can't operate on letters directly, so it uses the ID as a handle.

Embeddings

- Like translating each barcode into a **coordinate on a map**. Similar tokens end up near each other because the model learns a useful geometry for prediction.

Embedding matrix

- Like a giant **dictionary of learned "token fingerprints."** Index in, fingerprint out.

Context window (block size T)

- Like reading a paragraph through a **mail slot**: you only see the last *T* characters, so your guess is based on what fits through that slot.

Logits

- Like the model producing a **scoreboard** for every possible next token. Higher score = "I'm leaning toward this."

Softmax

- Like converting a scoreboard into **betting odds** that must sum to 1. It doesn't change the ranking, it just turns scores into probabilities.

Cross-entropy loss

- Like grading a multiple-choice test where you're penalized more if you were **confident and wrong**, and rewarded if you were confident and right.

Why "softmax + cross-entropy" simplifies nicely

- Like combining "convert scores to odds" + "grade the odds" into a single clean rule: **push probability toward the correct answer, pull it away from the others.**

Self-Attention (the core idea):

Self-attention overall

- Like each token asking: **"Which earlier tokens should I care about right now?"** and then building a custom summary from them.

Queries (Q)

- Like what a token is **looking for**. "Given what I am, what info do I want from the past?"

Keys (K)

- Like what each past token **advertises** about itself. "Here's what I contain."

Values (V)

- Like the actual **content** you'll copy/blend from. "If you choose me, here's what you get."

Q·K similarity scores

- Like "how well does my question match your label?" High match = "you're relevant to me."

Scaling by sqrt(H)

- Like keeping microphone volume from blowing out the speakers. Without scaling, dot products get too large and softmax becomes overly peaky.

Attention weights

- Like distributing a fixed budget of attention across earlier tokens: “30% you, 60% you, 10% you…”

Weighted sum (attention @ V)

- Like making a **playlist**: you don’t pick one song, you blend multiple songs at different volumes to create one vibe.

Causal mask (why it matters):

Causal mask

- Like taking a test where you’re not allowed to **peek at the answer key**. Future tokens are literally blocked from view so the task stays “predict the next token.”

Stability + depth:

Residual connections

- Like giving information a **highway bypass** around complex processing. If the new transformation isn’t helpful yet, the model can still pass the original signal forward.

“Residual helps gradients”

- Like adding an elevator next to a staircase: gradients can travel back without getting exhausted by too many nonlinear steps.

Layer Normalization

- Like setting all audio tracks to a consistent loudness **per token**, so training doesn't explode just because one layer got too "loud."

Why LN is per token

- Like leveling the volume for **each speaker individually**, not the whole room average.

Feedforward network (MLP):

Feedforward / MLP inside the block

- Like attention decides **what information to bring in**, and the MLP decides **how to transform it**.
- Another analogy: attention is "search," MLP is "editing."

ReLU

- Like a gate that says: "If this feature is helpful, let it through. If not, clamp it to zero."

Training mechanics:

Backpropagation

- Like running a factory in reverse to figure out **which machine caused the defect** and how much to adjust each knob.

Gradient

- Like the slope of a hill: it tells you **which direction increases loss**. You update parameters in the opposite direction to go downhill.

Learning rate

- Like step size when walking downhill: too big = you overshoot and fall, too small = you crawl forever.

SGD (stochastic gradient descent)

- Like adjusting your strategy based on **small random samples** of experience instead of reviewing your entire life before every decision.

Mini-batches

- Like taste-testing a soup with a few spoonfuls rather than drinking the whole pot before adding salt.

Overfitting

- Like memorizing the practice exam answers instead of learning the subject—great on that exact sheet, bad on anything new.

Shapes

(B, T, C)

- Like a bookshelf:
  - **B** = number of books (examples)
  - **T** = pages per book (time steps)
  - **C** = words per page (features / channels)

Matrix multiplications

- Like assembly lines: you're repeatedly "relabeling" and "mixing" information in consistent, mechanical ways.